Project Requirements

You will build a **Digital Zoo Manager** application. This program will allow a user to create different types of animals and add them to a "zoo," which will be managed in memory.

Part 1: The Class Hierarchy (Core Logic)

You must create a set of classes that model animals using inheritance.

1. **`Animal.java` (The Superclass):**
    - This will be the base class for all animals.
    - **Private Fields:** `name` (String), `age` (int).
    - **Constructor:** To initialize the name and age.
    - **Getters/Setters:** Public getters and setters for all fields.
    - **Methods:**
        - `makeSound()`: A method that returns a generic sound string, like `"The animal makes a sound."`.
        - `getInfo()`: A method that returns a string with the animal's name and age.
2. **`Mammal.java` (Subclass):**
    - This class must **extend** `Animal`.
    - **Private Field:** `furColor` (String).
    - **Constructor:** Must call the parent constructor (`super()`) and also initialize `furColor`.
    - **Getters/Setters:** For the `furColor` field.
    - **Method Overriding:**
        - Override the `makeSound()` method to return `"The mammal makes a sound."`.
        - Override the `getInfo()` method to include the fur color.
3. **`Bird.java` (Subclass):**
    - This class must **extend** `Animal`.
    - **Private Field:** `canFly` (boolean).
    - **Constructor:** Must call the parent constructor and initialize `canFly`.
    - **Getters/Setters:** For the `canFly` field.

- **Method Overriding:**
    - Override the `makeSound()` method to return `"The bird chirps."`.
    - Override the `getInfo()` method to include whether it can fly.

Part 2: The Graphical User Interface (GUI)

1. **`ZooFrame.java` (The View Class):**
    - Create the main window for the application using `JFrame`.
    - Use layout managers to organize your components neatly.
2. **Required GUI Components:**
    - **Input Form:**
        - Text fields (`JTextField`) for "Name" and "Age".
        - A way to select the animal type. A `JComboBox` with options like "Generic Mammal" and "Generic Bird" is a good choice.
        - A way to input the specific attribute (e.g., a text field for "Fur Color" or a `JCheckBox` for "Can Fly?").
    - **Action Buttons:**
        - An "Add Animal" button.
    - **Display Area:**
        - A `JTextArea` (non-editable) that acts as a log. When an animal is added, its information and sound should be appended to the text area.

Part 3: Application Logic

1. **Data Storage:**
    - In your `ZooFrame.java`, use an `ArrayList<Animal>` to store the animals you create. This demonstrates polymorphism, as your list can hold both `Mammal` and `Bird` objects.
2. **Functionality:**
    - The user fills out the form, selects an animal type, and provides its specific details.
    - When the "Add Animal" button is clicked:
        1. Your code should read the input from all the fields.
        2. Based on the selected type, it should create the correct object (either a new `Mammal` or a new `Bird`).

3. The new animal object is added to the `ArrayList<Animal>`.
4. The `JTextArea` is updated with a confirmation message by calling the new object's `getInfo()` and `makeSound()` methods.
    - *Example Log Entry:* `Added a new Mammal! Info: Name: Leo, Age: 5, Fur Color: Golden. Sound: The mammal makes a sound.`

Deliverables

- **Source Code:** The complete IntelliJ IDEA project folder, zipped. This must include all `.java` files (`Animal.java`, `Mammal.java`, `Bird.java`, `ZooFrame.java`, etc.).

How to answer:
- Create a new repository in your GitHub
- Git-push your answer at your new repository
- Send the repository URL as your answer

- Good Luck! ;)